
BAB 4

PEMROSESAN AWAL DATA

4.1 TUJUAN PERCOBAAN

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

1. Mengetahui permasalahan-permasalahan yang mungkin terdapat pada data mentah.
2. Mengimplementasikan teknik imputasi data.
3. Mengimplementasikan teknik visualisasi BoxPlot untuk analisis *outlier*.
4. Mengimplementasikan teknik normalisasi data.
5. Mengimplementasikan teknik *label encoding*.

4.2 ALAT-ALAT YANG DIGUNAKAN

Pada percobaan ini, akan digunakan berbagai alat dan bahan yang mendukung kegiatan praktikum. Alat-alat tersebut meliputi:

- Daftar alat utama: OS Windows/Linux/macOS, Anaconda, Google Colab
- Pustaka: pandas, numpy, matplotlib, seaborn

4.3 DASAR TEORI

Data yang tersedia mungkin memiliki beberapa permasalahan sehingga tidak dapat langsung digunakan oleh metode pembelajaran mesin (*machine learning*). Permasalahan-permasalahan yang sering dijumpai pada data, antara lain:

1. *Missing value*

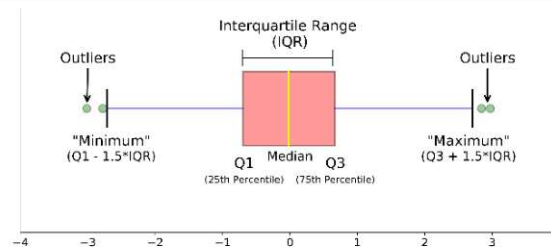
Missing value merupakan kondisi ketika sebuah data memiliki nilai yang hilang pada satu atau beberapa fiturnya. Hilangnya nilai pada suatu fitur tertentu pada sebuah *record* (baris) data menyebabkan *record* tersebut tidak dapat digunakan untuk proses pembelajaran mesin.

2. Adanya pencilan (*outlier*)

Outlier atau pencilan adalah data yang memiliki nilai yang sangat berbeda dari sebagian besar data lainnya dalam suatu *dataset*. *Outlier* bisa terjadi karena kesalahan input, variasi alami dalam data, atau fenomena yang memang tidak biasa. Kehadiran *outlier* dapat memengaruhi hasil analisis dan kinerja model *machine learning*. Contoh sederhana, diberikan data gaji 5.000, 2.000, 1.500, 3.000, dan 200.000 (*outlier*). Rata-rata gaji dengan *outlier* adalah 42.300, sedangkan apabila *outlier* dihapus maka rata-rata gajinya 2.875 (dan lebih masuk akal). Namun, penanganan *outlier* ini harus hati-hati dan perlu analisis lebih lanjut dalam kasus-kasus tertentu. Beberapa cara dapat dilakukan dengan menggunakan Interquartile Range, BoxPlot, Scatter Plot, Histogram, Z-score. Rumus untuk pencarian outlier dengan Interquartile Range adalah:

- **IQR** = $Q3 - Q1$ (selisih antara kuartil ke-3 dan kuartil ke-1)
- **Batas bawah** = $Q1 - 1.5 \times IQR$ (semua nilai kurang dari ini adalah *outlier*)
- **Batas atas** = $Q3 + 1.5 \times IQR$ (semua nilai lebih dari ini adalah *outlier*)

Visualisasi BoxPlot untuk mencari *outlier*:



Gambar 4.1 Contoh BoxPlot

3. Rentang data tidak sama

Sebuah data pada umumnya akan memiliki rentang data yang berbeda-beda pada setiap fiturnya. Hal ini dapat menimbulkan efek negatif pada metode pembelajaran mesin yang menggunakannya. Permasalahan timbul ketika suatu metode pembelajaran mesin menggunakan perhitungan jarak pada algoritmanya, karena rentang data yang lebih lebar akan mendominasi perhitungan jarak. Dengan kata lain, fitur yang memiliki rentang yang lebih sempit menjadi tidak terlalu berguna.

4. Inkonsistensi data

Inkonsistensi data terjadi apabila ada data yang tidak sesuai isinya dalam sebuah fitur khususnya apabila ada perbedaan sumber data. Sebagai contoh, tanggal dapat dituliskan dengan beberapa format dd/mm/yyyy, mm/dd/yyyy, yyyy-mm-dd, jenis kelamin ada yang menuliskan laki-laki dan perempuan, ada juga pria dan wanita, atau dituliskan singkatannya.

5. Representasi Data Kategori dan Ordinal

Fitur data kategori dan ordinal berisi data yang bukan numerik. Dalam kasus ini kita harus mengubahnya ke data numerik. Ada beberapa cara untuk mengubah data kategori dan ordinal:

- a. *One-hot Encoding* (OHE): teknik yang digunakan untuk mengubah data kategori menjadi representasi numerik biner. Dalam metode ini, setiap kategori dalam fitur dikonversi menjadi vektor dengan panjang sesuai jumlah kategori, hanya satu elemen bernilai 1 dan sisanya 0. Namun, hati-hati OHE dapat menyebabkan *curse of dimensionality* jika jumlah kategori sangat banyak, sehingga meningkatkan kompleksitas komputasi. Misalkan terdapat fitur "Warna" dengan tiga kategori: *Merah*, *Biru*, *Hijau*. Menggunakan *One-Hot Encoding*, data ini dikonversi menjadi:

Fitur Warna (Asli)	Hasil One-Hot Encoding		
	Merah	Biru	Hijau
Merah	1	0	0
Biru	0	1	0
Hijau	0	0	1

- b. *Label encoding*: mengubah kategori menjadi angka unik berdasarkan urutan tertentu. Setiap kategori diberikan nilai integer yang berbeda. Untuk fitur "Warna", kita dapat menerapkan Label Encoding sebagai berikut:

Fitur Warna (Asli)	Label Encoding
Merah	0
Biru	1
Hijau	2

Permasalahan pada data dapat diatasi pada tahapan *preprocessing* atau pemrosesan awal. Pada praktikum ini Anda akan mempelajari dua metode *preprocessing*, yaitu imputasi, normalisasi, dan *encoding*. Imputasi digunakan untuk mengatasi permasalahan *missing value*. Terdapat beberapa teknik untuk *mengganti* nilai yang hilang. Salah satunya adalah dengan mengisi nilai kosong dengan rata-rata data pada fitur dan kelas yang sama.

Normalisasi digunakan untuk mengatasi permasalahan rentang data yang berbeda-beda. Agar

rentang data pada masing-masing fitur menjadi sama atau hampir sama, dilakukanlah normalisasi. Terdapat dua metode normalisasi yang Anda pelajari pada praktikum ini, yaitu normalisasi MinMax dan Normalisasi Z-score. Persamaan normalisasi MinMax dan Z-score adalah:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)} \quad z = \frac{x - \mu}{\sigma}$$

Keterangan

x'	: Nilai ternormalisasi menggunakan minmax
x	: Nilai asal
$\min(x)$	: Nilai minimum pada suatu fitur
$\max(x)$	: Nilai maksimum pada suatu fitur
z	: Nilai ternormalisasi menggunakan z score
μ	: Rata-rata (<i>mean</i>) pada suatu fitur
σ	: Simpangan baku (<i>standard deviation</i>) pada suatu fitur

Perlu Anda perhatikan bahwa perhitungan **normalisasi dilakukan per fitur**. Dengan demikian, **nilai min, max, rata-rata, dan simpangan baku (*standard deviation*) dihitung per fitur**, bukan keseluruhan data.

4.4 PROSEDUR PERCOBAAN

4.4.1 IMPOR DATA

Unduh *dataset* yang akan digunakan pada praktikum kali ini. Anda dapat menggunakan aplikasi **wget** untuk mengunduh *dataset* dan menyimpannya dalam Google Colab. Jalankan *cell di bawah* ini untuk mengunduh *dataset*

```
! wget https://dataset-kal.s3.us-east-1.amazonaws.com/iris_missing.csv
```

Setelah *dataset* berhasil diunduh, langkah berikutnya adalah membaca *dataset* dengan memanfaatkan fungsi **readcsv** dari library pandas. Lakukan pembacaan berkas CSV menggunakan fungsi **readcsv**. Jangan lupa untuk melakukan import library pandas terlebih dahulu:

1. Percobaan Import Data

```
import pandas as pd
import numpy as np
data = pd.read_csv('iris_missing.csv')
```

Tampilkan beberapa baris dari *dataset* untuk mendapatkan informasi singkat mengenai isi data. Gunakan fungsi **head()** untuk menampilkan 5 data pertama.

```
data.head()
```

Berdasarkan informasi dari fungsi **head()**, data iris yang digunakan mempunyai 4 fitur sebagai berikut:

1. sepal length
2. sepal width
3. petal length
4. petal width

4.4.2 CEK TIPE DATA DAN STATISTIK SEDERHANA

Dataset yang dimuat ke *DataFrame* kadangkala masih perlu penyesuaian tipe data yang diinginkan, misalnya awalnya dari *integer* diubah menjadi *float*, bahkan ada *float16*, *float32*, *float64* (tergantung berapa *bit floating point*), *integer* pun juga memiliki perbedaan bit.

1. Percobaan Cek Tipe Data

Untuk memastikan tipe data dari masing-masing fitur, lakukan:

```
data.info()
```

Apabila ada ketidaksesuaian tipe data, kita dapat mengubahnya (*type casting*) secara manual, misalnya:

```
data['sepal_length'] = data['sepal_length'].astype(np.float32)
```

kemudian cek kembali dengan:

```
data.info()
```

Coba cek statistik data dengan menggunakan:

```
data.describe()
```

4.4.3 MISSING VALUE DAN IMPUTASI DATA

Jika Anda perhatikan dengan seksama, data pada baris ke-3 (indeks 2) pada fitur sepal length memiliki nilai 0.0. Hal ini menandakan adanya *missing value* pada data. Jalankan *cell di bawah* ini untuk mendapatkan semua data yang mengandung *missing value*. Pencarian data yang mengandung *missing value* dilakukan dengan tahapan sebagai berikut:

1. Percobaan Mengecek Missing Value dan Imputasi Data

1. Membuat filter untuk mencari data dengan sepal_length = 0, sepal_width = 0, petal_length = 0, petal_width = 0
2. Mencari data yang memenuhi *kondisi1* **atau** *kondisi2* **atau** *kondisi3* **atau** *kondisi4* menggunakan properti `loc` pada *DataFrame*.

```
kondisi1 = data['sepal_length']==0.0
kondisi2 = data['sepal_width']==0.0
kondisi3 = data['petal_length']==0.0
kondisi4 = data['petal_width']==0.0
data.loc[kondisi1 | kondisi2 | kondisi3 | kondisi4]
```

Penanganan *missing value* pada Pandas akan lebih mudah apabila data yang hilang (bernilai 0.0) diganti dengan *NaN* (*Not A Number*). Gunakan properti **replace** pada *DataFrame* untuk mengganti 0.0 menjadi *NaN*.

```
data = data.replace(0.0,np.NaN)
```

Terdapat beberapa cara untuk mengatasi permasalahan *missing value* pada data. Salah satu cara yang sederhana adalah dengan mengganti nilai *NaN* pada suatu fitur dengan rata-rata nilai fitur tersebut pada data lain yang bernilai bukan *NaN*. Perhatikan bahwa Anda harus mengganti nilai *NaN* dengan rata-rata data lain yang memiliki kategori (*species*) yang sama.

Fungsi-fungsi pada *pandas* yang dapat Anda manfaatkan:

1. **transform** untuk mengaplikasikan fungsi tertentu pada *DataFrame*, pada permasalahan ini fungsi yang digunakan adalah fungsi **mean**.
2. **groupby** untuk mengelompokkan *DataFrame* berdasarkan nilai kolom tertentu, pada permasalahan ini kolom yang digunakan adalah **species**.
3. **fillna** untuk mengganti nilai *NaN* dengan nilai yang telah ditentukan.

Buatlah fungsi bernama **imputasi** yang melakukan tahapan berikut:

1. Menghitung rata-rata masing-masing kolom berdasarkan kelasnya.
2. Mengisi kolom yang berisi *NaN* dengan rata-rata kelas yang sesuai.

```

1 def imputasi(df_input):
2     list_columns = df_input.columns #mendapatkan daftar kolom pada DataFrame
3     class_column = list_columns[-1] #kolom terakhir merupakan kolom kelas
4     for column in list_columns[:-1]:
5         # Penggantian nilai NaN dilakukan per kolom
6         df_input[column] = df_input[column].fillna(
            df_input.groupby(class_column)[column].transform('mean'))
7
8     return df_input

```

Buatlah sebuah *DataFrame* baru bernama **data_imputasi** yang berisi *dataset* dengan nilai *NaN* yang sudah diganti dengan cara memanggil fungsi **imputasi**.

```
data_imputasi = imputasi(data)
```

Cek apakah masih terdapat nilai *NaN* pada *DataFrame*.

```
data_imputasi.isnull().values.any()
```

Coba juga dengan:

```
data_imputasi.isna().any()
```

4.4.4 ANALISIS OUTLIER

Untuk mencari *outlier*, dalam percobaan ini kita akan gunakan BoxPlot untuk memvisualisasikan *outlier*. Mahasiswa juga diharapkan dapat mencari menggunakan kode Python.

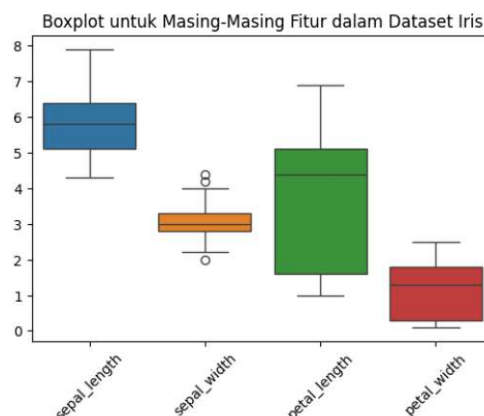
1. Percobaan Analisis Outlier Melalui Visualisasi

```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 plt.figure(figsize=(6, 4))
5 sns.boxplot(data=data_imputasi)
6 plt.xticks(rotation=45) # Memiringkan label sumbu x agar lebih terbaca
7 plt.title("Boxplot untuk Masing-Masing Fitur dalam Dataset Iris")
8 plt.show()

```

maka hasil *plotting* adalah:



Gambar 4.2 Contoh Visualisasi BoxPlot untuk Analisis Outlier

Dari grafik BoxPlot dapat kita lihat ada beberapa lingkaran di atas atau di bawah nilai ambang pada fitur *sepal_width*, inilah yang dikenal dengan *outlier*. Coba temukan dengan menggunakan kode Python dan lengkapi bagian yang kosong (titik-titik), apabila menggunakan hasil setelah imputasi ubah menjadi *data_imputasi* bukan *data*.

2. Percobaan Mencari Outlier

```

1 Q1 = data['sepal_width'].quantile(...) # Kuartil pertama (Q1)
2 Q3 = data['sepal_width'].quantile(...) # Kuartil ketiga (Q3)
3 IQR = Q3 - Q1 # Rentang Interkuartil
4
5 # Menentukan batas bawah dan atas untuk outlier
6 lower_bound = ... # Menentukan batas bawah outlier, gunakan rumus di dasar
  teori
7 upper_bound = ... # Menentukan batas atas outlier, gunakan rumus di dasar
  teori

```

Setelah dapat melakukan pencarian *outlier*, penghapusan *outlier* dapat dilakukan dengan cara memfilter titik-titik data yang memenuhi syarat misalnya:

```

data_without_outliers = data[~((data['sepal_width'] < lower_bound) |
  (data['sepal_width'] > upper_bound))]

```

4.4.5 NORMALISASI MINMAX

Normalisasi bertujuan menyamakan rentang nilai pada setiap fitur. Beberapa metode pembelajaran mesin memiliki kinerja yang buruk apabila rentang nilai tiap variabel berbeda jauh. Sebelum melakukan normalisasi, cek terlebih dahulu rentang (nilai max - nilai min) pada masing-masing fitur.

1. Percobaan Normalisasi Data

```

1 def cetak_rentang(df_input):
2     #mengambil nama kolom, kecuali yang terakhir (kelas)
3     list_fitur = df_input.columns[:-1]
4     for fitur in list_fitur:
5         max = df_input[fitur].max()
6         min = df_input[fitur].min()
7         print("Rentang fitur ", fitur, " adalah ", max-min)
8
9     cetak_rentang(data_imputasi)

```

Terlihat bahwa masing-masing fitur memiliki rentang yang berbeda, meskipun tidak terlalu signifikan. Pada praktikum ini Anda akan mengimplementasikan normalisasi MinMax untuk menyamakan rentang setiap fitur menjadi satu (1). Persamaan dari fungsi normalisasi MinMax adalah:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Implementasikan metode normalisasi MinMax pada fungsi bernama **minmax**.

```

1 def minmax(df_input):
2     list_fitur = df_input.columns[:-1]
3     for fitur in list_fitur:
4         max = df_input[fitur].max()
5         min = df_input[fitur].min()
6         df_input[fitur] = (df_input[fitur] - min) / (max - min)
7     return df_input

```

Buatlah sebuah *DataFrame* baru bernama **data_normal** yang berisi hasil dari metode minmax dengan input **data_imputasi**:

```

data_normal = minmax(data_imputasi)

```


Cek 5 baris pertama **data_normal**

```
data_normal.head()
```

Tampilkan rentang masing-masing fitur menggunakan fungsi **cetak_rentang** yang telah dibuat

```
cetak_rentang(data_normal)
```

4.4.6 LABEL ENCODING

Dalam kasus data yang diberikan, tidak ada data atau fitur kategori atau ordinal, namun ada label kelas (target) yaitu: *Iris-setosa*, *Iris-versicolor*, *Iris-virginica*. Untuk mengubah ke data numerik, khusus pada label kelas ini kita melakukan dengan label encoding, sedangkan pada fitur (*predictor*) kita dapat menggunakan *label encoding* atau *one-hot encoding*. Dalam contoh ini, akan digunakan *lambda function*. Buat fungsi yang akan dipanggil oleh *lambda function*.

```
1 def label_encoding(x):
2     if 'setosa' in x:
3         return 0
4     if 'versicolor' in x:
5         return 1
6     if 'virginica' in x:
7         return 2
```

Coba lakukan *encoding* pada target Species dari *DataFrame* hasil normalisasi sebelumnya:
`data_normal['Species'] = data_normal['Species'].apply(lambda x: label_encoding(x))`

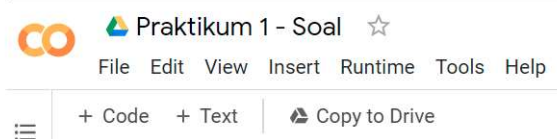
4.5 TUGAS

1. Implementasikan metode normalisasi Z-score dengan cara membuat fungsi bernama **zscore**.
2. Normalisasikan *DataFrame* **data_imputasi** menggunakan fungsi **zscore**. Simpan hasilnya pada *DataFrame* bernama **data_zscore**.
3. Jelaskan perbedaan hasil normalisasi **MinMax** dan **Z-Score**. Petunjuk: cek nilai rentang, rata-rata dan simpangan baku dari **data_zscore**.

Petunjuk pengerjaan soal:

1. Buka tautan berikut, pastikan Anda *login* menggunakan akun **Student UB**.
<https://s.ub.ac.id/nbkal04>

2. Klik tombol **Copy to Drive**



3. Beri nama file **Praktikum 4 - Nama - NIM.ipynb**
4. Gunakan tautan *dataset*: https://dataset-kal.s3.us-east-1.amazonaws.com/iris_missing_nan.csv sebagai tugas. Lakukan perubahan yang sesuai untuk mengatasi *missing value* pada *dataset* tersebut.
5. Isilah *cell* yang kosong, termasuk untuk pencarian dan penghapusan *outlier* (bagian yang dikosongkan di 4.4.4).
6. Download file ***.ipynb** dengan cara klik **File > Download .ipynb**
7. Kumpulkan file ***.ipynb** ke asisten

4.6 KESIMPULAN

Pemrosesan data dalam *machine learning* sangat penting untuk memastikan model yang dihasilkan akurat dan dapat diandalkan. Prinsip “*garbage in, garbage out*” (GIGO) menekankan bahwa jika data yang digunakan berkualitas buruk—misalnya mengandung nilai yang hilang, tidak relevan, atau tidak terstruktur dengan baik—maka model yang dihasilkan juga akan memiliki kinerja yang buruk. Oleh karena itu, teknik seperti imputasi, normalisasi, dan transformasi data diperlukan dalam tahap *preprocessing*. Imputasi digunakan untuk menangani nilai yang hilang dengan menggantinya menggunakan metode seperti mean, median, modus, atau model prediktif agar data tetap informatif. Normalisasi membantu menyelaraskan skala data, terutama dalam algoritma yang sensitif terhadap perbedaan skala seperti regresi dan jaringan saraf. Sementara itu, transformasi data mengubah format atau distribusi data agar lebih sesuai dengan model, seperti menggunakan normalisasi *MinMax*, *Z-score*, transformasi *log* untuk menangani distribusi yang sangat miring. Dengan menerapkan teknik *preprocessing* yang tepat, model *machine learning* dapat mempelajari pola dengan lebih baik dan menghasilkan prediksi yang lebih akurat.

Setelah melakukan semua percobaan ini maka mahasiswa diwajibkan membuat kesimpulan.

Kesimpulan dari percobaan ini merupakan bagian penting untuk merangkum temuan utama dan relevansinya dengan tujuan percobaan. Mahasiswa harus mampu menyimpulkan:

- Apakah tujuan percobaan telah tercapai?
- Hubungan antara hasil percobaan dan dasar teori.
- Faktor-faktor yang memengaruhi hasil percobaan, termasuk kemungkinan kesalahan yang terjadi.

Kesimpulan harus disusun secara singkat, jelas, dan berdasarkan data yang diperoleh.